Actividad Extra (Opcional)

Control de Intersecciones con Problemas de Satisfacción de Restricciones (CSP)

Contexto

En los sistemas de tráfico urbano, los semáforos deben coordinarse para evitar conflictos entre flujos de vehículos y al mismo tiempo maximizar la eficiencia del cruce.

Una forma alternativa de modelar este problema, además de heurísticas o aprendizaje por refuerzo, es utilizar **Problemas de Satisfacción de Restricciones (CSP)**.

En un CSP, se definen:

- **Variables**
- **Dominios posibles**
- **Restricciones entre variables**

El objetivo es encontrar una **asignación de valores que satisfaga todas las restricciones simultáneamente**.

Objetivo de la actividad

Modelar un **cruce semafórico simple** como un **problema de satisfacción de restricciones (CSP)** e implementar una solución en **Python**.

El modelo debe representar qué direcciones pueden tener luz verde **sin generar conflictos entre vehículos**.

Escenario base

Considera un cruce con cuatro direcciones:

```
          Norte
            |
Oeste ----+---- Este
            |
          Sur
```

Cada dirección puede:

- Avanzar
- Esperar

Los semáforos controlan los flujos.

Variables del CSP

Define variables como:

L_Norte
L_Sur
L_Este
L_Oeste

Cada variable representa el **estado del semáforo en esa dirección**.

Dominios posibles

Cada variable puede tomar valores:

{Rojo, Verde}

Restricciones

Algunas restricciones típicas son:

Restricción 1

Norte y Sur pueden estar verdes al mismo tiempo.

L_Norte = Verde
L_Sur = Verde

Restricción 2

Este y Oeste pueden estar verdes al mismo tiempo.

L_Este = Verde
L_Oeste = Verde

Restricción 3

No puede haber flujo cruzado simultáneo.

Ejemplo:

Si L_Norte = Verde → L_Este = Rojo

Restricción 4

Al menos una dirección debe tener verde.

Lo que deben implementar

Los estudiantes deben crear un **modelo CSP en Python** que:

1. Defina variables
2. Defina dominios
3. Defina restricciones
4. Encuentre combinaciones válidas

Sugerencia de implementación

Pueden usar:

Opción 1 (recomendada)

Implementar **backtracking manual**

Ejemplo simplificado:

```
variables = ["N", "S", "E", "W"]
domain = ["R", "G"]

def is_valid(assignment):
```

```
    if assignment["N"] == "G" and assignment["E"] == "G":
        return False

    return True
```

Opción 2

Usar librería:

python-constraint

Extensión al reto del curso

Una vez encontrado un estado válido del semáforo, los estudiantes deben:

1. Integrar esa configuración a su **modelo del cruce**
2. Simular qué flujo vehicular se permite

Esto conecta directamente con su **proyecto de intersecciones**.

Análisis requerido

Cada estudiante debe responder brevemente:

1️⃣ ¿Qué variables definiste?
2️⃣ ¿Cuáles fueron las restricciones del sistema?
3️⃣ ¿Cuántas configuraciones válidas encontraste?
4️⃣ ¿Cómo se compara este enfoque con heurísticas o Q-learning?

Entregable

Un **notebook de Python o Google Colab** que incluya:

- Definición del CSP
- Código funcional
- Resultados

- Breve análisis (5–10 líneas)

Nombre del archivo (Incluye tu matricula):

CSP_Traffic_<matricula>.ipynb

Evaluación sugerida (10 puntos extra)

Criterio	Puntos
Modelado correcto del CSP	3
Implementación en Python	3
Resultados válidos	2
Análisis breve	2